CONTRAT_GEN

Documentation technique

Architecture, fonctionnement interne et maintenance

Pour l'installateur et le développeur

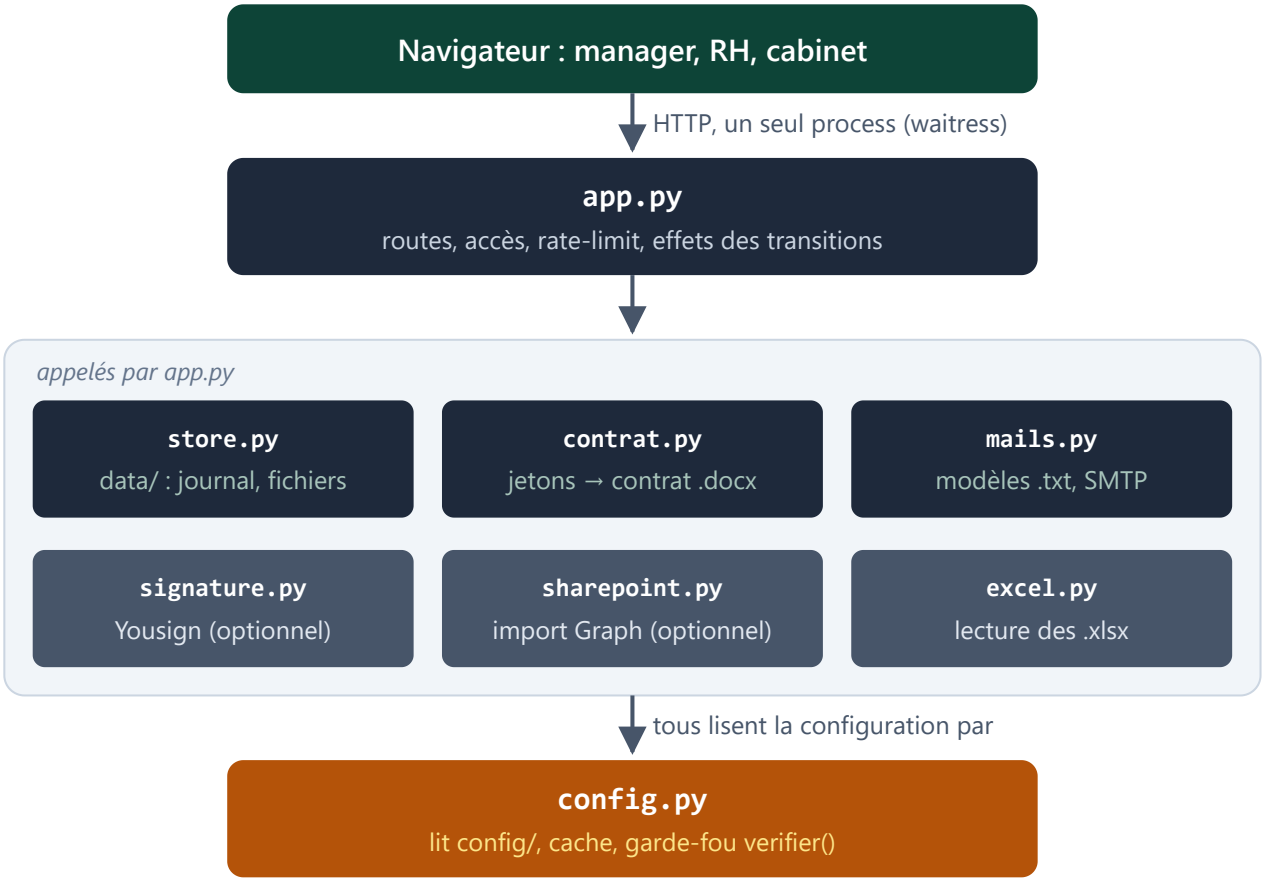
Édition du 13 octobre 2026

Sommaire

1. En bref
2. Le parcours d'un dossier
3. La configuration
4. La génération des contrats
5. Mails et sécurité
6. L'import des salariés déjà en poste
7. Installer, lancer, tester

1. En bref

Contrat_Gen reçoit les demandes d'embauche des managers, aide la RH à produire le contrat et la DPAE, puis transmet le dossier au cabinet comptable.



Les scripts (installer, configurer, doctor, recap, purger…) appellent les mêmes modules, sans app.py.

Les modules et ce qu'ils font

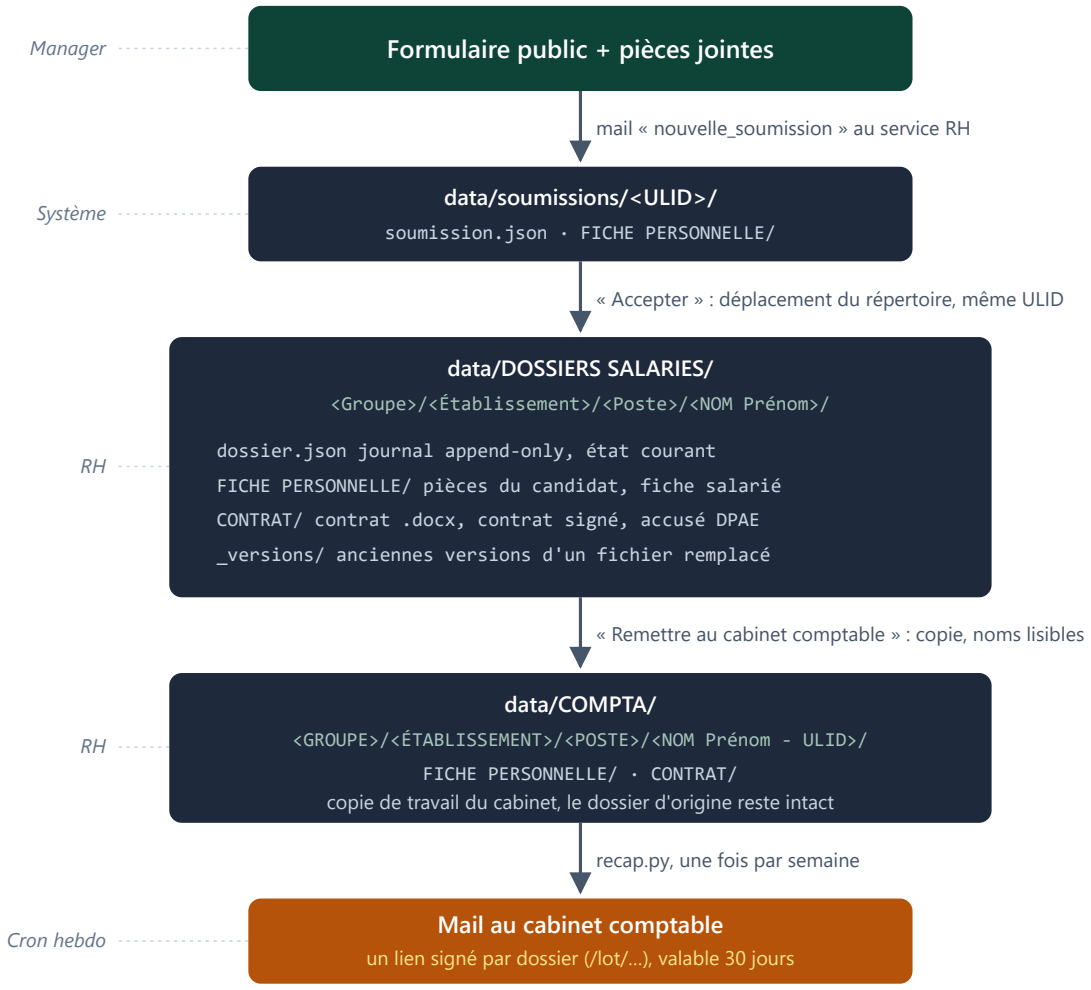
Cinq règles expliquent presque tout le code :

- **Un client = une copie du dépôt.** Chaque client a sa propre configuration (`config/`) et ses propres données (`data/`). Rien n'est partagé.

- **Pas de base de données.** Les dossiers sont des répertoires et des fichiers JSON sur le disque.
- **Un seul process.** Les verrous sont en mémoire : deux serveurs sur les mêmes données perdraient des écritures. Ne jamais lancer plusieurs workers.
- **La configuration ne contient pas de code.** Tout ce qui change d'un client à l'autre est dans `config/`, en JSON et en modèles de documents.
- **L'application refuse de démarrer** si la configuration est incomplète, et dit pourquoi.

---

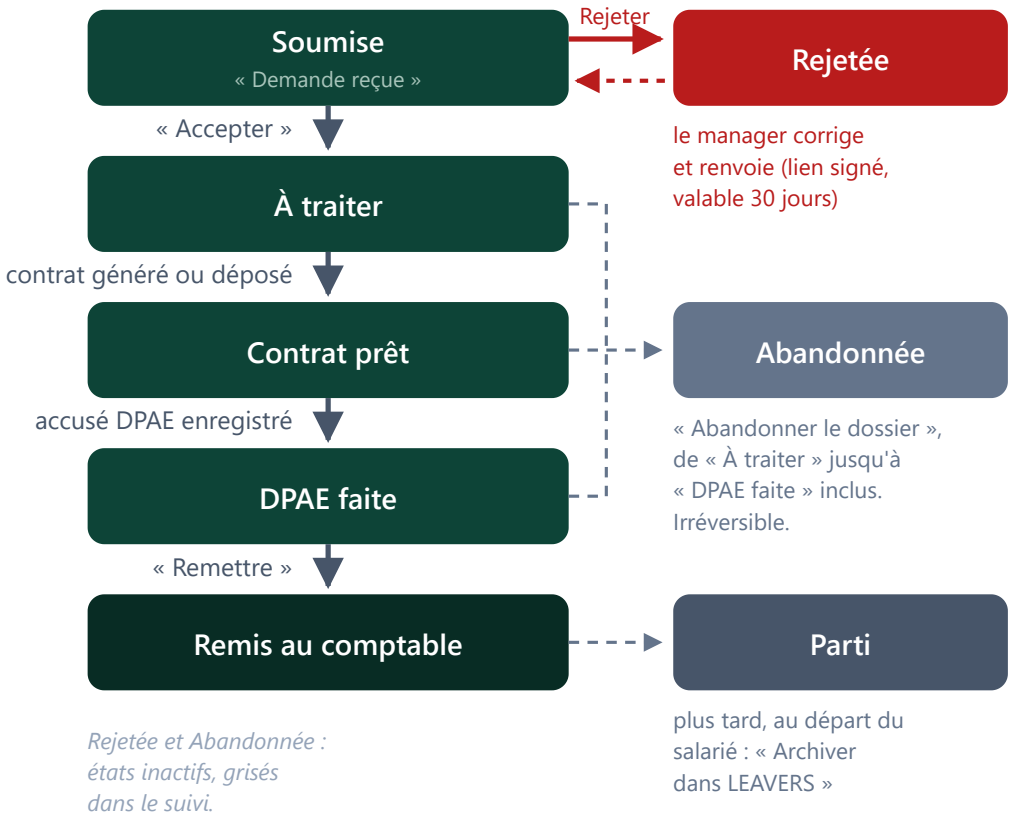
2. Le parcours d'un dossier



Aucune base de données : le disque est la base, dossier.json fait foi sur l'état.

Du formulaire public au cabinet comptable

Un dossier passe par des **états** : demande reçue, à traiter, contrat prêt, DPAE faite, remis au comptable. Il peut aussi être rejeté (le manager corrige et renvoie) ou abandonné, jusqu'à la DPAE. Quand le salarié quitte l'entreprise, son dossier passe à « parti ».



Les états d'un dossier

Chaque dossier a un fichier `dossier.json` avec un **journal** : une ligne par action. Aucune ligne n'est jamais supprimée ; la purge retire seulement les commentaires et les adresses IP. L'état du dossier, c'est simplement la dernière ligne du journal.

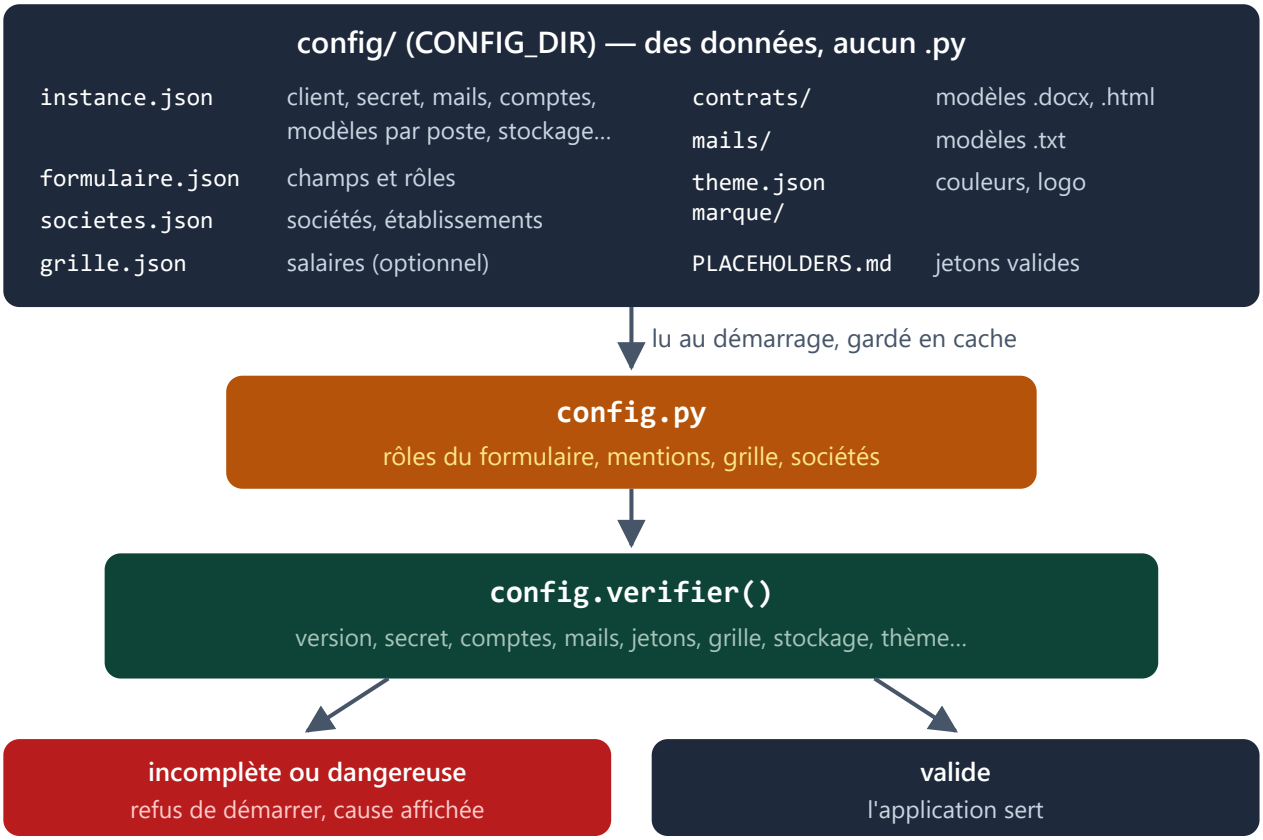
```
{"de": "ContratPret", "vers": "DpaeFaite", "le": "2026-09-16T12:35:24+00:00", "par": "rh"}
```

Ce qu'il faut savoir pour modifier ce code :

- Les changements d'état passent par `store.transition()`, qui refuse un passage non prévu. Quatre fonctions font exception, car elles créent ou déplacent le dossier : `valider()`, `rejeter()`, `resoumettre()` et `importer()`.
- Certaines lignes ne changent pas l'état : elles notent un événement (mail raté, fiche générée…). On les écrit avec `store.noter()`.
- Le reste du stockage est dans `data/` : `.serveur-actif.json` empêche deux serveurs de tourner sur les mêmes données, `recap.json` retient le dernier mail hebdomadaire envoyé, `mails/` reçoit les mails en mode test.

---

3. La configuration



Les fichiers de config/ et la vérification au démarrage

`config.py` lit ces fichiers une fois, puis les garde en mémoire. Pour appliquer une modification, il faut **redémarrer**.

Les trois fichiers principaux :

- `instance.json` : le client, ses adresses mail, ses comptes RH, et **quel modèle de contrat utiliser pour quel poste** (`templates` : la première règle qui correspond gagne).
- `formulaire.json` : les champs du formulaire. Un champ peut alimenter un `{{Jeton}}` des contrats. Chaque pièce jointe doit avoir un `role`, qui sert à nommer le fichier.
- `societes.json` : les sociétés et leurs établissements, avec les mentions légales du contrat.

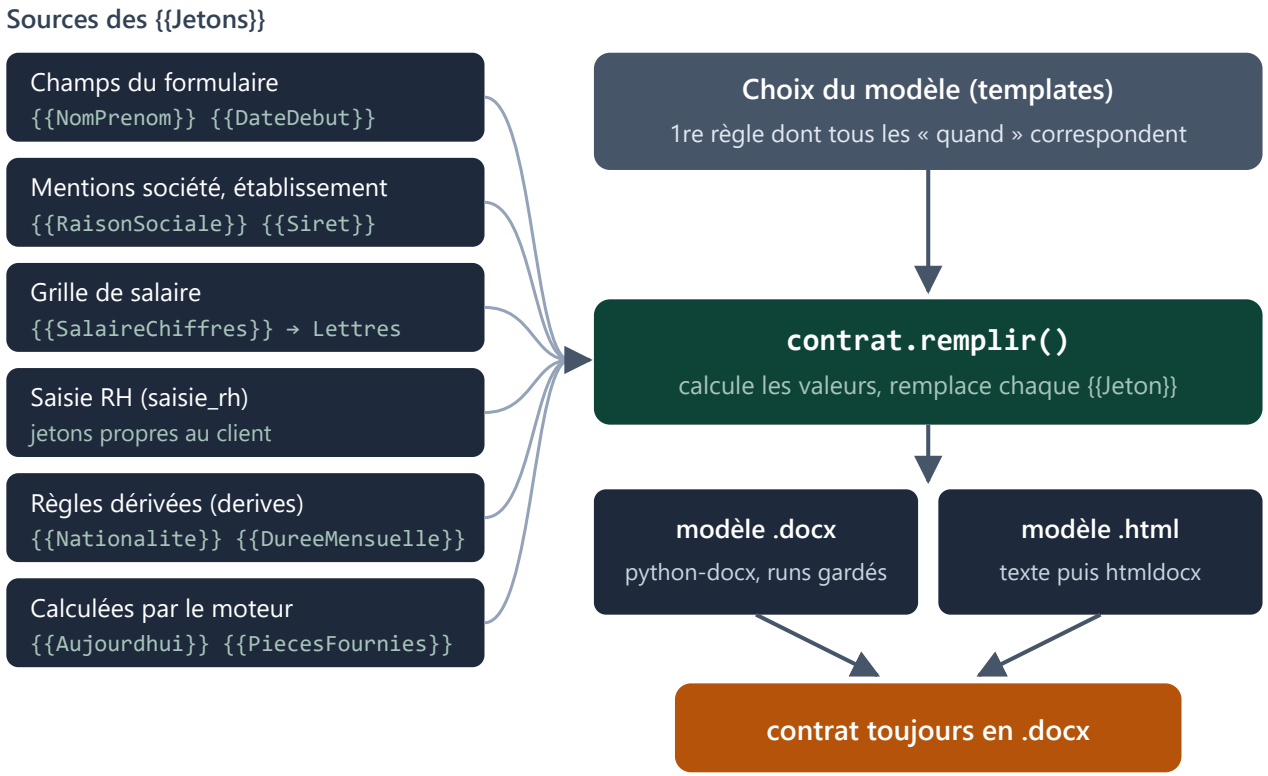
Un exemple complet de chaque fichier se trouve dans `config.demo/`.

Sociétés et établissements dans Supabase

C'est facultatif. Si `instance.json` contient un bloc `supabase`, les sociétés sont lues dans la base, et `societes.json` devient une simple copie de secours. Si la base ne répond pas, l'application utilise cette copie.

La clé du bloc donne un accès complet à la base : elle ne doit jamais aller dans Git. Les tables à créer sont dans `docs/supabase-schema.sql`.

4. La génération des contrats



Contrôles : config.verifier() refuse au démarrage un jeton sans source ;
doctor.py génère un contrat pour chaque croisement (poste, temps partiel, établissement…) ;
placeholders.py écrit la liste des jetons valides (PLACEHOLDERS.md).

D'où viennent les valeurs, et comment le modèle devient un .docx

Un modèle de contrat est un fichier `.docx` ou `.html` qui contient des jetons comme `{{NomPrenom}}`. `contrat.remplir()` remplace chaque jeton par sa valeur. Le contrat produit est toujours un `.docx`.

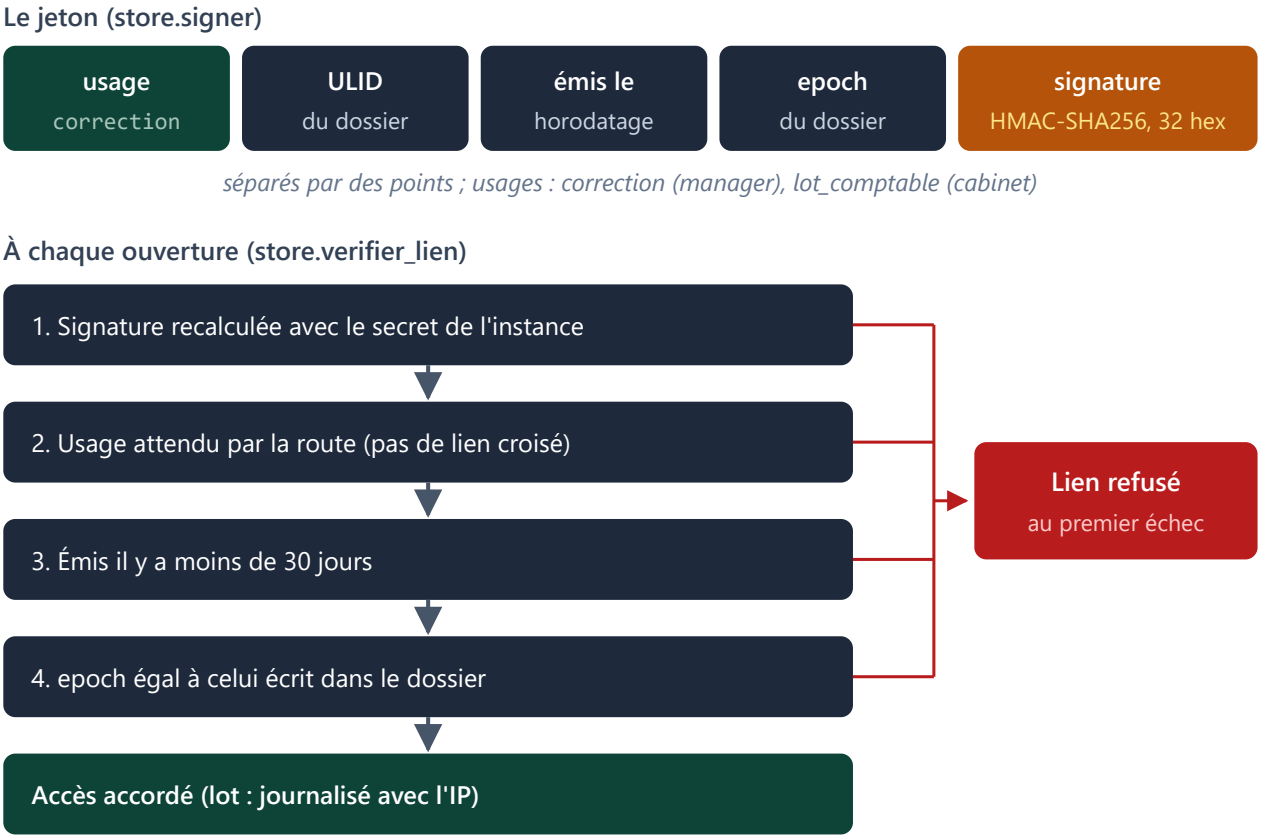
Trois outils évitent les mauvaises surprises :

- **Au démarrage**, un jeton qui n'a pas de source bloque le lancement.
- `python doctor.py` génère un contrat d'essai pour chaque combinaison possible (poste, temps partiel, établissement…) et signale les trous.
- `python placeholders.py` écrit `PLACEHOLDERS.md`, la liste des jetons utilisables, à donner au client qui prépare ses modèles.

5. Mails et sécurité

Les mails sont des fichiers texte dans `config/mails/`. Un mail qui ne part pas n'arrête jamais l'action en cours : l'échec est noté dans le journal du dossier et affiché à l'écran. Le mail de rejet va à l'établissement, ou à la RH si l'établissement n'a pas d'adresse.

Le manager et le cabinet comptable n'ont pas de compte. Ils reçoivent un **lien signé**, valable 30 jours :



Révoquer = incrémenter l'epoch : link_epoch à chaque rejet ; lien_comptable_epoch au renvoi, à l'abandon, à la purge.

Comment un lien signé est construit et vérifié

Les autres protections :

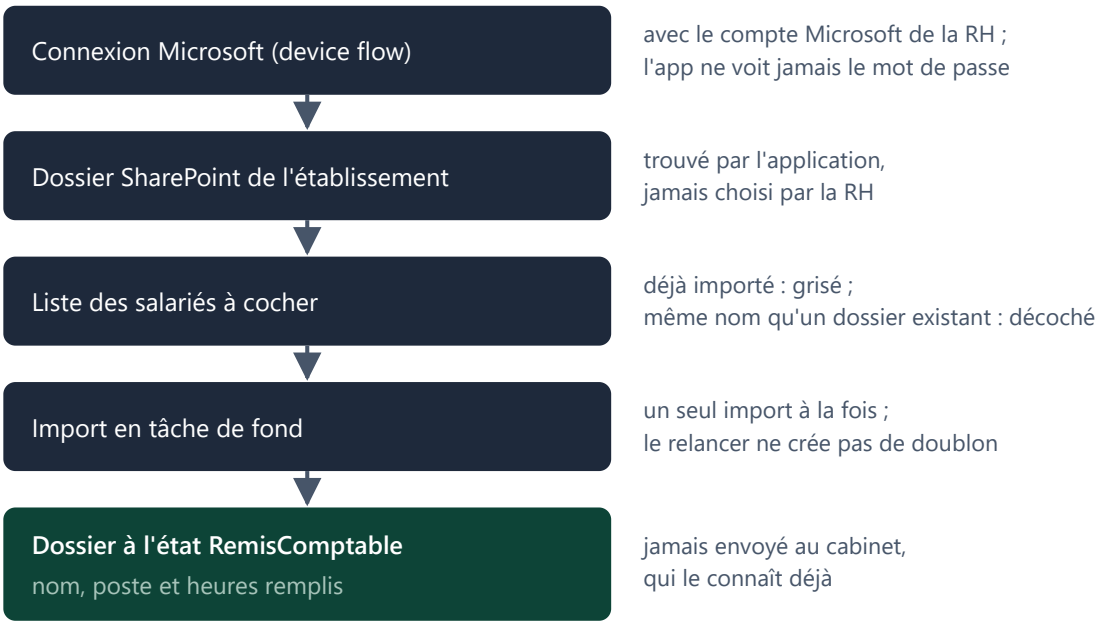
- Les comptes RH sont dans `instance.json`, avec des mots de passe hachés.
- Le formulaire et la connexion limitent le nombre d'essais par adresse IP.
- Les fichiers envoyés sont limités en type (PDF, images) et en taille, et renommés par l'application.

---

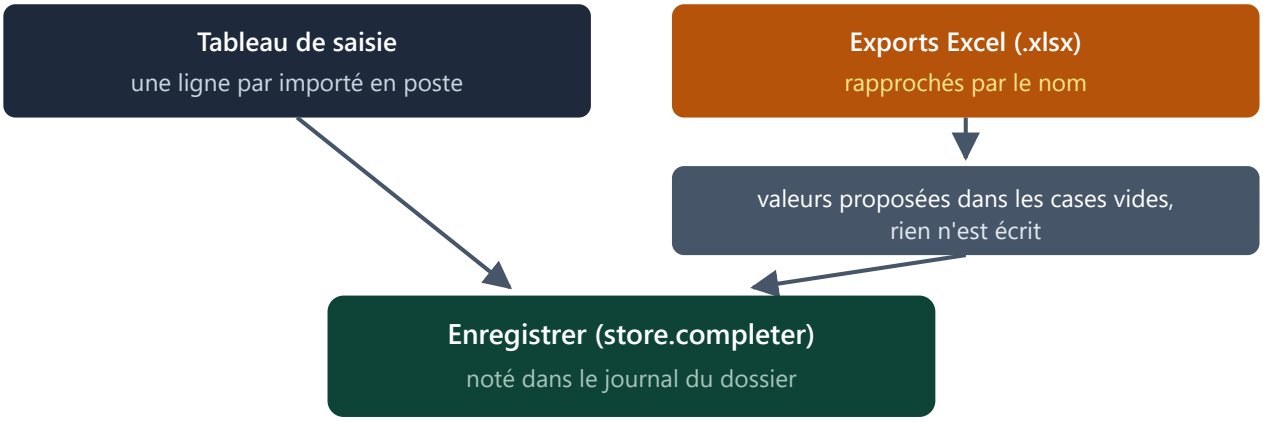
6. L'import des salariés déjà en poste

Les salariés embauchés avant l'outil ont déjà un dossier sur le SharePoint du client. Deux écrans les font entrer dans l'application. Ils n'existent que si `instance.json` contient un bloc `sharepoint`.

1. Récupérer les dossiers (sharepoint.py)



2. Compléter les dossiers (excel.py)

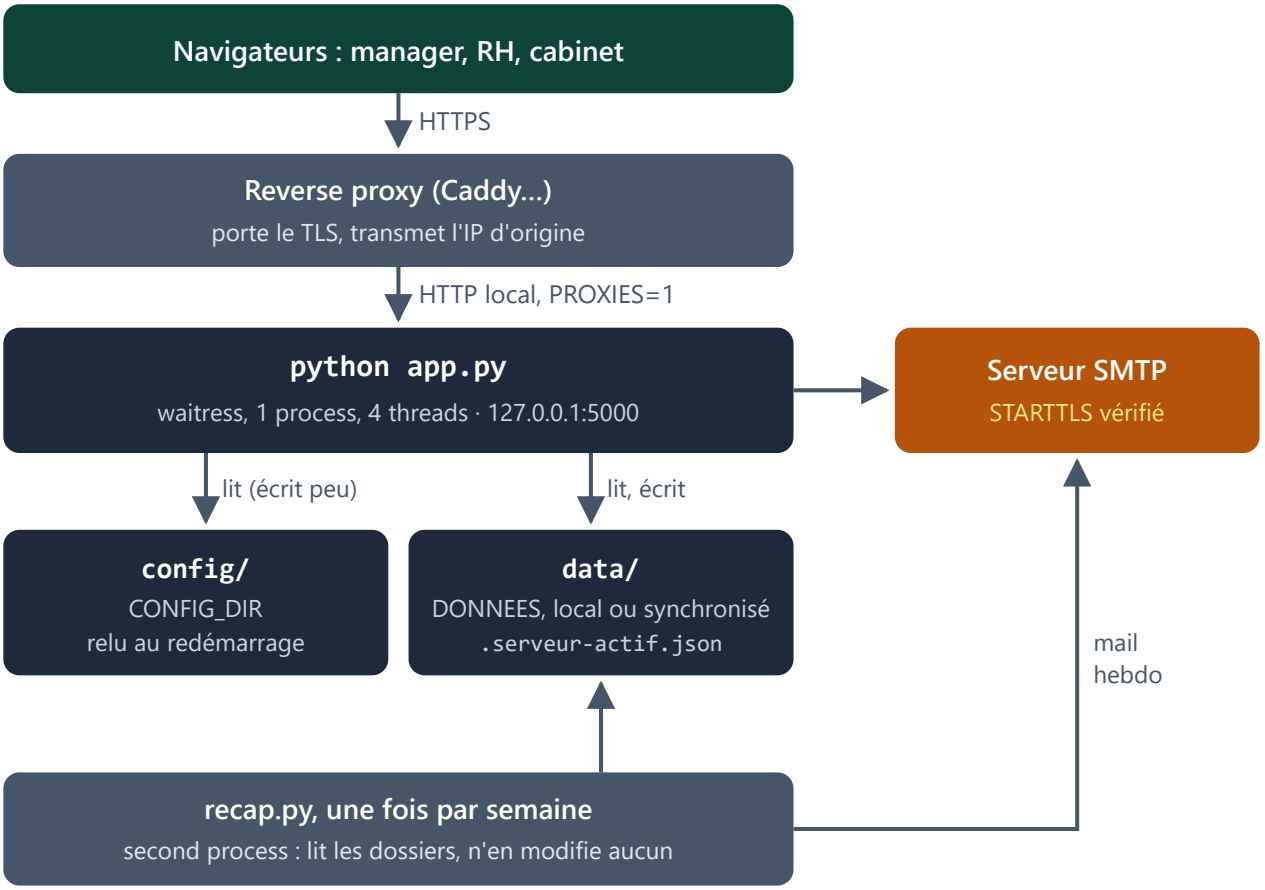


Récupérer puis compléter les dossiers

À retenir :

- La RH se connecte avec son propre compte Microsoft. L'application ne voit jamais son mot de passe. Il faut se déconnecter après l'import.
- Un dossier importé arrive directement à l'état « remis au comptable », mais il n'est **jamais** envoyé au cabinet, qui le connaît déjà.
- Relancer un import ne crée pas de doublon.
- L'Excel ne fait que **proposer** des valeurs. Rien n'est enregistré tant que la RH n'a pas cliqué sur « Enregistrer ».
- Les modules concernés : `sharepoint.py` (import) et `excel.py` (lecture des fichiers Excel). Il faut aussi installer `requirements-sharepoint.txt`.

7. Installer, lancer, tester



Un seul serveur par stockage : le verrou refuse un second démarrage, même depuis une autre machine.

Une instance en production

```
pip install -r requirements.txt
python installer.py "Basilic Café" /srv/basilic   # crée la configuration du client
export CONFIG_DIR=/srv/basilic/config DONNEES=/srv/basilic/data
python configurer.py      # dit ce qu'il manque dans la configuration
python doctor.py          # vérifie les modèles de contrat
PROXIES=1 python app.py   # lance le serveur derrière un reverse proxy
```

- `DEBUG=1 python app.py` lance le serveur de développement.
- `recap.py` envoie le mail hebdomadaire au cabinet. Il doit être planifié chaque semaine (cron ou tâche Windows), sinon le cabinet ne reçoit rien.
- `PROXIES` indique combien de reverse proxies sont devant l'application, pour lire la vraie adresse IP des visiteurs.
- Pour sauvegarder, il suffit de copier `config/` et `data/`. Pour mettre à jour, `git pull` puis redémarrer.

Tests. Chaque fichier de `tests/` se lance seul (`python tests/test_parcours.py`). La suite est bonne quand tous terminent sans erreur. Si vous ajoutez un fichier `.py` à la racine, ajoutez-le aussi dans `tests/test_clients.py` .